Capital University

[formerly Helwan University]

Faculty of Computers & Artificial Intelligence

Mainstream Program

CS 316 Algorithms

Spring Semester 2 — 2025/2026

Algorithm Analysis Report

Problem 11: Longest Subarray with Equal 0s and 1s

Problem Statement

Given a binary array (containing only 0s and 1s), find the length of the longest contiguous subarray that contains an equal number of 0s and 1s.

Examples

Input Array	Output	Explanation
[0, 1]	2	Entire array has one 0 and one 1
[0, 1, 0]	2	Subarray [0,1] or [1,0] length 2
[0, 0, 1, 0, 1, 1, 0]	6	Subarray [0,0,1,0,1,1] — three 0s and three 1s

Key Solution

Replace every 0 with -1 and every 1 with +1. A subarray with equal 0s and 1s is exactly one whose transformed elements sum to 0. This transformation connects all three solutions below.

Solution 1 — Brute Force (Nested Loops)

1.1 Pseudocode

```
ALGORITHM MaxLenBruteForce(arr[0..n-1])
// Finds the longest subarray with equal 0s and 1s
// Input: A binary array arr of size n
// Output: Length of the longest balanced subarray

res ← 0

for s ← 0 to n-1 do
    sum ← 0
    for e ← s to n-1 do
        if arr[e] = 0
            sum ← sum + (-1)
        else
            sum ← sum + 1

        if sum = 0
            res ← max(res, e - s + 1)

return res
```

1.2 C++ Implementation

```
// program to find the longest subarray with
// equal number of 0s and 1s using nested loop

#include <iostream>
#include <vector>
using namespace std;

int maxLen(vector<int> &arr) {
    int res = 0;

    // Pick a starting point as 's'
    for (int s = 0; s < arr.size(); s++) {
        int sum = 0;

        // Consider all subarrays arr[s...e]
        for (int e = s; e < arr.size(); e++) {
             sum += (arr[e] == 0) ? -1 : 1;

                    // Check if it's a 0-sum subarray
            if (sum == 0)
                    // update max size
                res = max(res, e - s + 1);
        }
    }
    return res;
```

```
}

int main() {
    vector<int> arr = { 1, 0, 0, 1, 0, 1, 1 };
    vector<int> arr2 = {0,1,0};
    vector<int> arr3 = {0,1};
    cout << maxLen(arr) << endl;   // 6
    cout << maxLen(arr2) << endl;  // 2
    cout << maxLen(arr3) << endl;  // 2
}
```

1.3 Analysis

Step 1 — Decide on Input Size

The input size is n = the number of elements in the binary array arr[0..n-1].

Step 2 — Identify the Basic Operation

The basic operation is the sum update inside the inner loop:

sum ← sum + (-1 or +1)

It executes exactly once per inner-loop iteration and dominates the running time.

Step 3 — Does C(n) Depend on Input Type?

Both loops always run their full range regardless of array contents — the inner loop never breaks early. Therefore C(n) depends only on n, not on the specific input values.

-> This means best case = worst case for the loop count.

Step 4 — Set Up the Sum

The outer loop runs from s = 0 to n-1. For each value of s, the inner loop runs from e = s to n-1. The number of basic operation executions is:

$$c(n) = \sum_{s=0}^{n-1} \sum_{e=s}^{n-1} 1$$

Step 5 — Solve the Inner Sum First

For a fixed value of s, the inner loop runs from e = s to n-1:

$$\sum_{e=s}^{n-1} 1 = (n-1) - s + 1 = n - s$$

Step 6 — Solve the Outer Sum

Substituting into the outer sum:

Now substitute into the outer sum:

$$C(n) = \sum_{s=0}^{n-1} (n - s)$$

Expand the terms:

$$C(n) = n + (n-1) + (n-2) + \cdots + 1$$

This is the sum of the first $\boldsymbol{n}$ *positive integers:*

$$C(n) = \sum_{s=0}^{n-1} (n - s) = \frac{n(n+1)}{2}$$

Therefore:

$$C(n) = \frac{n(n+1)}{2} = \frac{n^2 + n}{2} \approx \frac{1}{2} n^2$$

Using the standard formula from the appendix (Formula 2):

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} \approx \frac{1}{2} n^2$$

.

1.4 Complexity

Best Case

The two loops always complete fully — no early exit exists. The minimum work is still the full double loop:

$$\boldsymbol{C_{best}(n) = \frac{n(n+1)}{2}}$$

$$\boldsymbol{\therefore C_{best}(n) \in \Theta(n^2)}$$

Worst Case

Same reasoning — loops never terminate early:

$$\boldsymbol{\therefore C_{worst}(n) \in \Theta(n^2)}$$

Case	T(n)	Complexity
Best Case	$n(n+1)/2$	$\Theta(n^2)$
Worst Case	$n(n+1)/2$	$\Theta(n^2)$
Average Case	$n(n+1)/2$	$\Theta(n^2)$

➔ Since there is no early termination, all three cases are identical — the algorithm always performs $\Theta(n^2)$ operations.

Solution 2 — Recursive Version

2.1 Pseudocode

```
ALGORITHM maxLen(a, i, sum, mp, len)
// Input : array a[0..n-1], index i, running prefix-sum,
// hash map mp{sum→first_index}, best length len
// Output: length of longest equal-0s-1s subarray

  n ← a.size
  if i = n // base case: all elements processed
    return len
  if a[i] = 0 then sum ← sum − 1
            else sum ← sum + 1
  if sum ∉ mp // first time seeing this prefix sum
    mp[sum] ← i
  else // same prefix sum seen before → equal segment found
    len ← max(len, i − mp[sum])
  return maxLen(a, i+1, sum, mp, len) // tail call
```

2.2 C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

int maxLen(vector<int> a, int i = 0, int sum = 0,
           unordered_map<int, int> mp = {{0, -1}}, int len = 0)
{
    int n = a.size();
    if (i == n)
        return len;    // base case

    sum += a[i] == 0 ? -1 : 1;       // O(1)
    if (mp.find(sum) == mp.end())    // O(1)
        mp[sum] = i;                // O(1)
    else
        len = max(len, i - mp[sum]);  // O(1)

    return maxLen(a, i + 1, sum, mp, len);  // one recursive call
}

int main() {
    vector<int> arr = { 1, 0, 0, 1, 0, 1, 1 };
    vector<int> arr2 = {0,1,0};
    vector<int> arr3 = {0,1};
    cout << maxLen(arr) << endl;   // 6
    cout << maxLen(arr2) << endl;  // 2
    cout << maxLen(arr3) << endl;  // 2
}
```

2.3 Analysis

Step 1 — Decide on Input Size

The input size is n = the number of elements in the binary array arr[0..n-1].

n = a.size()

Step 2 — Identify the Basic Operation

The basic operation is the **sum update** — it executes exactly once per call (after the base-case check), regardless of the array contents:

```
            // basic operation — runs once per call
                  sum += a[i] == 0 ? -1 : 1;

      // these are also O(1) but not the "counted" operation
         if (mp.find(sum) == mp.end()) // O(1) hash lookup

                mp[sum] = i;
         else
                len = max(len, i - mp[sum]); // O(1)
```

The sum update is chosen because it is the *core computation* the algorithm exists to perform

Step 3 — Set Up the Recurrence

Each call processes one element (index i) and makes one recursive call on the remaining **n − 1** elements. The base case is when i == n — nothing left to process.

```
          // one recursive call, shrinks by 1 each time
             return maxLen(a, i + 1, sum, mp, len);
```

T(n) = T(n−1) + 1 for n > 0
T(0) = 0 base case: i == n, return immediately
maxLenRecursive: T(n) = T(n−1) + 1, T(0) = 0

Step 4 — Solve the recurrence — iteration method

Expand by substituting the recurrence into itself repeatedly:

T(n) = T(n−1) + 1
= T(n−2) + 1 + 1 substitute T(n−1)
= T(n−2) + 2
= T(n−3) + 3 substitute T(n−2)
⋮
= T(n−k) + k

The base case is reached when n – k = 0 → k = n, we then have:

T(n) = T(n−n) + n
= T(0) + n
= 0 + n
= n

Therefore the method maxLenRecursive is O(n)

Iteration method solution

T(n) = T(n−1) + c

= T(n−2) + c + c = T(n−2) + 2c

= T(n−3) + 3c

⋮

= T(n−k) + k·c

Base case reached when n − k = 0 ⟶ k = n

T(n) = T(0) + n·c = Θ(1) + n·c

∴ T(n) = Θ(n)

2.7 Complexity

Time complexity :

Θ(n)

n recursive calls, O(1) each

Space complexity :

Θ(n)

hash map + call stack depth n

.

Solution 3 — Optimized (Hash Map + Prefix Sum)

3.1 Pseudocode

```
ALGORITHM MaxLenOptimized(arr[0..n-1])
// Finds the longest subarray with equal 0s and 1s
// using prefix sum and hash map
// Input: A binary array arr of size n
// Output: Length of the longest balanced subarray

mp ← empty hash map
preSum ← 0
res ← 0

for i ← 0 to n-1 do
    if arr[i] = 0
        preSum ← preSum + (-1)
    else
        preSum ← preSum + 1

    if preSum = 0
        res ← i + 1

    if preSum exists in mp
        res ← max(res, i - mp[preSum])
    else
        mp[preSum] ← i

return res
```

3.2 C++ Implementation

```
// program to find longest subarray with equal
// number of 0's and 1's using Hashmap and Prefix Sum
#include <iostream>
#include <vector>
#include <unordered_map>
using namespace std;

int maxLen(vector<int> &arr) {
    unordered_map<int, int> mp;

    int preSum = 0;
    int res = 0;

    // Traverse through the given array
    for (int i = 0; i < arr.size(); i++) {

        // Add current element to sum
        // if current element is zero, add -1
        preSum += (arr[i] == 0) ? -1 : 1;
```

```
        // To handle sum = 0 at last index
        if (preSum == 0)
            res = i + 1;

        // If this sum is seen before, then update
        // result with maximum
        if (mp.find(preSum) != mp.end())
            res = max(res, i - mp[preSum]);

        // Else put this sum in hash table
        else
            mp[preSum] = i;
    }

    return res;
}

int main() {
    vector<int> arr = { 1, 0, 0, 1, 0, 1, 1 };
    vector<int> arr2 = {0,1,0};
    vector<int> arr3 = {0,1};
    cout << maxLen(arr) << endl;   // 6
    cout << maxLen(arr2) << endl;  // 2
    cout << maxLen(arr3) << endl;  // 2
}
```

3.3 Analysis

Step 1 — Decide on Input Size

The input size is n = the number of elements in the binary array arr[0..n-1].

Step 2 — Identify the Basic Operation

The basic operation is the prefix sum update inside the single loop:

```
preSum  <-  preSum + (-1  or  +1)
```

This executes exactly once per iteration. Hash map operations (find and insert) run in Θ(1) average time and do not change the order of growth.

Step 3 — Does C(n) Depend on Input Type?

There is only a **single loop** from i = 0 to n−1 with no early termination. Every element is visited exactly once regardless of the array's content. Therefore C(n) depends **only on n**, not on the type of input.

This means best case = worst case for the loop count.

.

Step 4 — Set Up the Sum

The single loop runs from i = 0 to n−1. The basic operation executes once per iteration:

$$C(n) = \sum_{i=0}^{n-1} 1$$

Step 5 — Solve the Sum

Applying Formula 1 :

$$\sum_{i=l}^{u} 1 = u - l + 1$$

Substituting l = 0, u = n−1:

$$C(n) = \sum_{i=0}^{n-1} 1 = (n-1) - 0 + 1 = n$$

Therefore:

$$C(n) = n$$

3.4 Complexity

Best Case

$$\therefore\therefore C_{best}(n) \in \Theta(n)$$

Worst Case

$$\therefore C_{worst}(n) \in \Theta(n)$$

Case	T(n)	Complexity
Best Case	n	Θ(n)
Worst Case	n	Θ(n)
Average Case	n	Θ(n)

➔ Since there is no early termination and every element is visited exactly once, all three cases are identical — the algorithm always performs Θ(n) operations.

Overall Comparison of All Three Solutions

Algorithm	Technique	Time Complexity	Space Complexity
Brute Force (Nested Loops)	Iterative	$\Theta(n^2)$	$\Theta(1)$
Recursive Version	Recursion	$\Theta(n)$	$\Theta(n)$
Optimized (Hash Map)	Iterative + Hash Map	$\Theta(n)$	$\Theta(n)$

Key Observations

1. The Brute Force iterative version is $\Theta(n^2)$ in time because it uses nested loops to enumerate all $O(n^2)$ subarrays and check each one.
2. The current Recursive version is $\Theta(n)$ in time — it makes exactly one pass through the array, processing each element once, identical in work to the optimized iterative approach.
3. The Recursive version uses $\Theta(n)$ extra space for two reasons: the call stack grows n frames deep (one per element), and the hash map mp stores at most n+1 prefix sum entries. Unlike a purely iterative version, the call stack space cannot be avoided here.
4. The Recursive solution achieves $\Theta(n)$ time by exploiting the prefix sum property: if the same prefix sum appears at two indices i and j, then the subarray arr[i+1..j] has equal 0s and 1s. The hash map mp records the first occurrence of each prefix sum in O(1) time, passed along through every recursive call.
5. Compared to the brute force, the current recursive solution trades $\Theta(n)$ extra space (call stack + hash map) in exchange for reducing time from $\Theta(n^2)$ to $\Theta(n)$ — a classic time-space tradeoff, achieved recursively rather than iteratively.